```
// ================= EXP 5 =================
#include <stdio.h>
#include <stdlib.h>
int mutex = 1;
int full = 0;
int empty; // Buffer size will be taken from user
int x = 0;
int wait(int s)
{
   return (--s);
}
int signal(int s)
{
   return (++s);
}
void producer()
{
   mutex = wait(mutex);
   empty = wait(empty);
   full = signal(full);
   x++;
   printf("\nProducer produced item %d", x);
   mutex = signal(mutex);
}
void consumer()
{
   mutex = wait(mutex);
   full = wait(full);
   empty = signal(empty);
   printf("\nConsumer consumed item %d", x);
   x--;
```

```
   mutex = signal(mutex);
}
int main()
{
   int choice;
   printf("Enter Buffer Size: ");
   scanf("%d", &empty); // Taking buffer size from user
   while (1)
   {
      printf("\n\n1. Producer\n2. Consumer\n3. Exit");
      printf("\nEnter your choice: ");
      scanf("%d", &choice);
      switch (choice)
      {
         case 1:
            if ((mutex == 1) && (empty != 0))
               producer();
            else
               printf("\nBuffer is Full!");
            break;
         case 2:
            if ((mutex == 1) && (full != 0))
               consumer();
            else
               printf("\nBuffer is Empty!");
            break;
         case 3:
            exit(0);
         default:
            printf("\nInvalid Choice!");
      }   }   return 0; }
```

```
// ================= EXP 6 =================
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define N 5
#define EAT_COUNT 3 // Number of times each philosopher eats
sem_t chopstick[N];
pthread_t philos[N];
void *philosopher(void *num)
{
  int id = *(int *)num;
  for(int i = 0; i < EAT_COUNT; i++)
  {
    printf("Philosopher %d is Thinking\n", id);
    sleep(1);
    sem_wait(&chopstick[id]); // Left chopstick
    sem_wait(&chopstick[(id + 1) % N]); // Right chopstick
    printf("Philosopher %d is Eating (%d)\n", id, i+1);
    sleep(2);
    sem_post(&chopstick[id]); // Release left
    sem_post(&chopstick[(id + 1) % N]); // Release right
  }
  printf("Philosopher %d has finished eating.\n", id);
  pthread_exit(NULL);
}
int main()
{
  int i;
  int phil_num[N];
```

```
    for (i = 0; i < N; i++)
        sem_init(&chopstick[i], 0, 1);
    for (i = 0; i < N; i++)
    {
        phil_num[i] = i;
        pthread_create(&philos[i], NULL, philosopher, &phil_num[i]);
    }
    for (i = 0; i < N; i++)
        pthread_join(philos[i], NULL);
    printf("\nAll philosophers have finished eating.\n");
    return 0;
}
```

```
// ================= EXP 7 =================
#include <stdio.h>
int main()
{
   int n, m, i, j, k;
   printf("Enter number of processes: ");
   scanf("%d", &n);
   printf("Enter number of resource types: ");
   scanf("%d", &m);
   int alloc[n][m], max[n][m], need[n][m];
   int avail[m], finish[n], safeSeq[n];
   printf("\nEnter Allocation Matrix:\n");
   for(i = 0; i < n; i++)
      for(j = 0; j < m; j++)
         scanf("%d", &alloc[i][j]);
   printf("\nEnter Max Matrix:\n");
   for(i = 0; i < n; i++)
      for(j = 0; j < m; j++)
         scanf("%d", &max[i][j]);
   printf("\nEnter Available Resources:\n");
   for(i = 0; i < m; i++)
      scanf("%d", &avail[i]);
   for(i = 0; i < n; i++)
      for(j = 0; j < m; j++)
         need[i][j] = max[i][j] - alloc[i][j];
   for(i = 0; i < n; i++)
      finish[i] = 0;
   int count = 0;
   while(count < n)
   {
      int found = 0;
```

```
  for(i = 0; i < n; i++)
  {
    if(finish[i] == 0)
    {
      int flag = 1;
      for(j = 0; j < m; j++)
      {
        if(need[i][j] > avail[j])
        {
          flag = 0;
          break;
        }
      }
      if(flag)
      {
        for(k = 0; k < m; k++)
          avail[k] += alloc[i][k];
        safeSeq[count++] = i;
        finish[i] = 1;
        found = 1;
      }
    }
  }
  if(found == 0)
  {
    printf("\nSystem is in Unsafe State!");
    return 0;
  }
}
printf("\nSystem is in Safe State.\nSafe Sequence: ");
for(i = 0; i < n; i++) {    printf("P%d ", safeSeq[i]); }   return 0;}
```

```
// ================= EXP 8 =================
// ---- First Fit ----
#include <stdio.h>
int main() {
   int blockSize[10], originalBlock[10], processSize[10];
   int blockCount, processCount;
   int allocation[10];
   int totalMemory = 0, totalAllocated = 0, totalFree = 0;
   printf("Enter number of memory blocks: ");
   scanf("%d", &blockCount);
   printf("Enter size of each block:\n");
   for(int i = 0; i < blockCount; i++) {
      scanf("%d", &blockSize[i]);
      originalBlock[i] = blockSize[i]; // Store original size
      totalMemory += blockSize[i]; // Calculate total memory
   }
   printf("Enter number of processes: ");
   scanf("%d", &processCount);
   printf("Enter size of each process:\n");
   for(int i = 0; i < processCount; i++) {
      scanf("%d", &processSize[i]);
      allocation[i] = -1;
   }
   // First Fit Allocation
   for(int i = 0; i < processCount; i++) {
      for(int j = 0; j < blockCount; j++) {
         if(blockSize[j] >= processSize[i]) {
            allocation[i] = j;
            blockSize[j] -= processSize[i];
            totalAllocated += processSize[i];
            break;
```

```
        }
      }
    }
    // Display Allocation
    printf("\nProcess No.\tProcess Size\tBlock No.\n");
    for(int i = 0; i < processCount; i++) {
      printf("%d\t\t%d\t\t", i+1, processSize[i]);
      if(allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
      else
        printf("Not Allocated\n");
    }
    // Calculate remaining free memory
    for(int i = 0; i < blockCount; i++) {
      totalFree += blockSize[i];
    }
    printf("\nTotal Memory = %d", totalMemory);
    printf("\nTotal Allocated Memory = %d", totalAllocated);
    printf("\nTotal Free Memory = %d\n", totalFree);
    return 0;
}
// ---- Best Fit ----
#include <stdio.h>
int main() {
    int blockSize[10], originalBlock[10], processSize[10];
    int blockCount, processCount;
    int allocation[10];
    int totalMemory = 0, totalAllocated = 0, totalFree = 0;
    printf("Enter number of memory blocks: ");
    scanf("%d", &blockCount);
    printf("Enter size of each block:\n");
```

```
for(int i = 0; i < blockCount; i++) {
   scanf("%d", &blockSize[i]);
   originalBlock[i] = blockSize[i]; // store original size
   totalMemory += blockSize[i]; // calculate total memory
}
printf("Enter number of processes: ");
scanf("%d", &processCount);
printf("Enter size of each process:\n");
for(int i = 0; i < processCount; i++) {
   scanf("%d", &processSize[i]);
   allocation[i] = -1; // no allocation initially
}
// Best Fit Allocation
for(int i = 0; i < processCount; i++) {
   int bestIndex = -1;
   for(int j = 0; j < blockCount; j++) {
      if(blockSize[j] >= processSize[i]) {
         if(bestIndex == -1 || blockSize[j] < blockSize[bestIndex]) {
            bestIndex = j;
         }
      }
   }
   if(bestIndex != -1) {
      allocation[i] = bestIndex;
      blockSize[bestIndex] -= processSize[i];
      totalAllocated += processSize[i];
   }
}
printf("\nProcess No.\tProcess Size\tBlock No.\n");
for(int i = 0; i < processCount; i++) {
   printf("%d\t\t%d\t\t", i+1, processSize[i]);
```

```
        if(allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
    for(int i = 0; i < blockCount; i++) {
        totalFree += blockSize[i];
    }
    printf("\nTotal Memory = %d", totalMemory);
    printf("\nTotal Allocated Memory = %d", totalAllocated);
    printf("\nTotal Free Memory = %d\n", totalFree);
    return 0;
}
// ---- Worst Fit ----
#include <stdio.h>
int main() {
    int blockSize[10], originalBlock[10], processSize[10];
    int blockCount, processCount;
    int allocation[10];
    int totalMemory = 0, totalAllocated = 0, totalFree = 0;
    printf("Enter number of memory blocks: ");
    scanf("%d", &blockCount);
    printf("Enter size of each block:\n");
    for(int i = 0; i < blockCount; i++) {
        scanf("%d", &blockSize[i]);
        originalBlock[i] = blockSize[i]; // store original size
        totalMemory += blockSize[i]; // calculate total memory
    }
    printf("Enter number of processes: ");
    scanf("%d", &processCount);
    printf("Enter size of each process:\n");
```

```
for(int i = 0; i < processCount; i++) {
   scanf("%d", &processSize[i]);
   allocation[i] = -1; // initially not allocated
}
// Worst Fit Allocation
for(int i = 0; i < processCount; i++) {
   int worstIndex = -1;
   for(int j = 0; j < blockCount; j++) {
      if(blockSize[j] >= processSize[i]) {
         if(worstIndex == -1 || blockSize[j] > blockSize[worstIndex]) {
            worstIndex = j;
         }
      }
   }
   if(worstIndex != -1) {
      allocation[i] = worstIndex;
      blockSize[worstIndex] -= processSize[i];
      totalAllocated += processSize[i];
   }
}
printf("\nProcess No.\tProcess Size\tBlock No.\n");
for(int i = 0; i < processCount; i++) {
   printf("%d\t\t%d\t\t", i+1, processSize[i]);
   if(allocation[i] != -1)
      printf("%d\n", allocation[i] + 1);
   else
      printf("Not Allocated\n");
}
for(int i = 0; i < blockCount; i++) {
   totalFree += blockSize[i];
}
```

```
    printf("\nTotal Memory = %d", totalMemory);
    printf("\nTotal Allocated Memory = %d", totalAllocated);
    printf("\nTotal Free Memory = %d\n", totalFree);
    return 0;
}
```

```
// ================= EXP 9 =================
// -------- FIFO --------
#include <stdio.h>
int main() {
   int frames[10], pages[50];
   int frameCount, pageCount;
   int i, j, k;
   int pageFaults = 0;
   int index = 0; // FIFO pointer
   int hit;
   printf("Enter number of frames: ");
   scanf("%d", &frameCount);
   printf("Enter number of pages: ");
   scanf("%d", &pageCount);
   printf("Enter page reference string:\n");
   for(i = 0; i < pageCount; i++) {
      scanf("%d", &pages[i]);
   }
   // Initialize frames as empty
   for(i = 0; i < frameCount; i++) {
      frames[i] = -1;
   }
   printf("\nStep\tPage\tFrames\t\tStatus\n");
   printf("-------------------------------------------------\n");
   for(i = 0; i < pageCount; i++) {
      hit = 0;
      // Check for page hit
      for(j = 0; j < frameCount; j++) {
         if(frames[j] == pages[i]) {
            hit = 1;
            break;
```

```
            }
        }
        // If page fault
        if(hit == 0) {
            frames[index] = pages[i];
            index = (index + 1) % frameCount;
            pageFaults++;
        }
        // Print step details
        printf("%d\t%d\t", i+1, pages[i]);
        for(k = 0; k < frameCount; k++) {
            if(frames[k] != -1)
                printf("%d ", frames[k]);
            else
                printf("- ");
        }
        if(hit)
            printf("\tHit\n");
        else
            printf("\tFault\n");
    }
    printf("\nTotal Page Faults = %d\n", pageFaults);
    printf("Total Page Hits = %d\n", pageCount - pageFaults);
    return 0;
}
//-------- LRU --------
#include <stdio.h>
int main() {
    int frames[10], pages[50], time[10];
    int frameCount, pageCount;
    int i, j, k;
```

```
int pageFaults = 0;
int counter = 0;
int hit, pos, min;
printf("Enter number of frames: ");
scanf("%d", &frameCount);
printf("Enter number of pages: ");
scanf("%d", &pageCount);
printf("Enter page reference string:\n");
for(i = 0; i < pageCount; i++) {
   scanf("%d", &pages[i]);
}
// Initialize frames
for(i = 0; i < frameCount; i++) {
   frames[i] = -1;
   time[i] = 0;
}
printf("\nStep\tPage\tFrames\t\tStatus\n");
printf("-------------------------------------------------\n");
for(i = 0; i < pageCount; i++) {
   hit = 0;
   // Check for page hit
   for(j = 0; j < frameCount; j++) {
      if(frames[j] == pages[i]) {
         counter++;
         time[j] = counter;
         hit = 1;
         break;
      }
   }
   // If page fault
   if(hit == 0) {
```

```
    pos = -1;
    // Check for empty frame
    for(j = 0; j < frameCount; j++) {
      if(frames[j] == -1) {
        pos = j;
        break;
      }
    }
    // If no empty frame, replace LRU
    if(pos == -1) {
      min = time[0];
      pos = 0;
      for(j = 1; j < frameCount; j++) {
        if(time[j] < min) {
          min = time[j];
          pos = j;
        }
      }
    }
    counter++;
    frames[pos] = pages[i];
    time[pos] = counter;
    pageFaults++;
  }
  // Print current frames
  printf("%d\t%d\t", i+1, pages[i]);
  for(k = 0; k < frameCount; k++) {
    if(frames[k] != -1)
      printf("%d ", frames[k]);
    else
      printf("- ");
```

```
        }
        if(hit)
            printf("\tHit\n");
        else
            printf("\tFault\n");
    }
    printf("\nTotal Page Faults = %d\n", pageFaults);
    printf("Total Page Hits = %d\n", pageCount - pageFaults);
    return 0;
}
// -------- Optimal --------
#include <stdio.h>
int main() {
    int frames[10], pages[50];
    int frameCount, pageCount;
    int i, j, k;
    int pageFaults = 0;
    int hit, pos, farthest, index;
    printf("Enter number of frames: ");
    scanf("%d", &frameCount);
    printf("Enter number of pages: ");
    scanf("%d", &pageCount);
    printf("Enter page reference string:\n");
    for(i = 0; i < pageCount; i++) {
        scanf("%d", &pages[i]);
    }
    // Initialize frames
    for(i = 0; i < frameCount; i++) {
        frames[i] = -1;
    }
    printf("\nStep\tPage\tFrames\t\tStatus\n");
```

```
printf("-------------------------------------------------\n");
for(i = 0; i < pageCount; i++) {
  hit = 0;
  // Check for page hit
  for(j = 0; j < frameCount; j++) {
    if(frames[j] == pages[i]) {
      hit = 1;
      break;
    }
  }
  // If page fault
  if(hit == 0) {
    pos = -1;
    for(j = 0; j < frameCount; j++) {
      if(frames[j] == -1) {
        pos = j;
        break;
      }
    }
    if(pos == -1) {
      farthest = -1;
      for(j = 0; j < frameCount; j++) {
        int nextUse = -1;
        for(k = i + 1; k < pageCount; k++) {
          if(frames[j] == pages[k]) {
            nextUse = k;
            break;
          }
        }
        if(nextUse == -1) {
          pos = j;
```

```
                break;
            }
            if(nextUse > farthest) {
                farthest = nextUse;
                pos = j;
            }
          }
        }
        frames[pos] = pages[i];
        pageFaults++;
    }
    printf("%d\t%d\t", i+1, pages[i]);
    for(j = 0; j < frameCount; j++) {
        if(frames[j] != -1)
            printf("%d ", frames[j]);
        else
            printf("- ");
    }
    if(hit)
        printf("\tHit\n");
    else
        printf("\tFault\n");
  }
  printf("\nTotal Page Faults = %d\n", pageFaults);
  printf("Total Page Hits = %d\n", pageCount - pageFaults);
  return 0;
}
```

```
// ================= EXP 10 =================
// -------- FCFS --------
#include <stdio.h>
#include <stdlib.h>
int main()
{
   int n, i, head, total = 0;
   printf("Enter number of requests: ");
   scanf("%d", &n);
   int req[n];
   printf("Enter the request sequence:\n");
   for(i = 0; i < n; i++)
   {
      scanf("%d", &req[i]);
   }
   printf("Enter initial head position: ");
   scanf("%d", &head);
   for(i = 0; i < n; i++)
   {
      total = total + abs(head - req[i]);
      head = req[i];
   }
   printf("Total head movement = %d\n", total);
   return 0;
}
// -------- SCAN --------
#include <stdio.h>
#include <stdlib.h>
int main()
{
   int req[50], n, head, total = 0, i, j, temp, dir;
```

```
printf("Enter number of requests: ");
scanf("%d",&n);
printf("Enter request sequence:\n");
for(i=0;i<n;i++)
   scanf("%d",&req[i]);
printf("Enter initial head position: ");
scanf("%d",&head);
printf("Enter direction (0=left,1=right): ");
scanf("%d",&dir);
for(i=0;i<n-1;i++)
   for(j=i+1;j<n;j++)
      if(req[i] > req[j])
      {
         temp=req[i];
         req[i]=req[j];
         req[j]=temp;
      }
int index;
for(i=0;i<n;i++)
   if(head < req[i])
   {
      index=i;
      break;
   }
if(dir==1)
{
   for(i=index;i<n;i++)
   {
      total += abs(head-req[i]);
      head=req[i];
   }
```

```
      total += abs(head-199);
      head=199;
      for(i=index-1;i>=0;i--)
      {
         total += abs(head-req[i]);
         head=req[i];
      }
   }
   else
   {
      for(i=index-1;i>=0;i--)
      {
         total += abs(head-req[i]);
         head=req[i];
      }
      total += abs(head-0);
      head=0;
      for(i=index;i<n;i++)
      {
         total += abs(head-req[i]);
         head=req[i];
      }
   }
   printf("Total head movement = %d\n",total);
   return 0;
}
// -------- C-SCAN --------
#include <stdio.h>
#include <stdlib.h>
int main()
{
```

```
int req[50], n, head, total = 0, i, j, temp;
printf("Enter number of requests: ");
scanf("%d",&n);
printf("Enter request sequence:\n");
for(i=0;i<n;i++)
   scanf("%d",&req[i]);
printf("Enter initial head position: ");
scanf("%d",&head);
for(i=0;i<n-1;i++)
   for(j=i+1;j<n;j++)
      if(req[i] > req[j])
      {
         temp=req[i];
         req[i]=req[j];
         req[j]=temp;
      }
int index;
for(i=0;i<n;i++)
   if(head < req[i])
   {
      index=i;
      break;
   }
for(i=index;i<n;i++)
{
   total += abs(head - req[i]);
   head = req[i];
}
total += abs(head - 199);
head = 199;
total += abs(head - 0);
```

```
    head = 0;
    for(i=0;i<index;i++)
    {
        total += abs(head - req[i]);
        head = req[i];
    }
    printf("Total head movement = %d\n", total);
    return 0;
}
// -------- LOOK --------
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int req[50], n, head, total = 0, i, j, temp, dir;
    printf("Enter number of requests: ");
    scanf("%d",&n);
    printf("Enter request sequence:\n");
    for(i=0;i<n;i++)
        scanf("%d",&req[i]);
    printf("Enter initial head position: ");
    scanf("%d",&head);
    printf("Enter direction (0=left,1=right): ");
    scanf("%d",&dir);
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(req[i] > req[j])
            {
                temp=req[i];
                req[i]=req[j];
                req[j]=temp;
```

```
    }
int index;
for(i=0;i<n;i++)
  if(head < req[i])
  {
    index=i;
    break;
  }
if(dir==1)
{
  for(i=index;i<n;i++)
  {
    total += abs(head-req[i]);
    head=req[i];
  }
  for(i=index-1;i>=0;i--)
  {
    total += abs(head-req[i]);
    head=req[i];
  }
}
else
{
  for(i=index-1;i>=0;i--)
  {
    total += abs(head-req[i]);
    head=req[i];
  }
  for(i=index;i<n;i++)
  {
    total += abs(head-req[i]);
```

```
        head=req[i];
      }
    }
    printf("Total head movement = %d\n",total);
    return 0;
}
// -------- C-LOOK --------
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int req[50], n, head, total = 0, i, j, temp;
    printf("Enter number of requests: ");
    scanf("%d",&n);
    printf("Enter request sequence:\n");
    for(i=0;i<n;i++)
      scanf("%d",&req[i]);
    printf("Enter initial head position: ");
    scanf("%d",&head);
    for(i=0;i<n-1;i++)
      for(j=i+1;j<n;j++)
        if(req[i] > req[j])
        {
          temp=req[i];
          req[i]=req[j];
          req[j]=temp;
        }
    int index;
    for(i=0;i<n;i++)
      if(head < req[i])
      {
```

```
            index=i;
            break;
        }
    for(i=index;i<n;i++)
    {
        total += abs(head-req[i]);
        head = req[i];
    }
    if(index > 0)
    {
        total += abs(head - req[0]);
        head = req[0];
    }
    for(i=1;i<index;i++)
    {
        total += abs(head - req[i]);
        head = req[i];
    }
    printf("Total head movement = %d\n", total);
    return 0;
}
// -------- SSTF --------
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int req[50], visited[50];
    int n, head, total = 0, i, j, index, min;
    printf("Enter number of requests: ");
    scanf("%d",&n);
    printf("Enter request sequence:\n");
```

```
    for(i=0;i<n;i++)
    {
        scanf("%d",&req[i]);
        visited[i] = 0;
    }
    printf("Enter initial head position: ");
    scanf("%d",&head);
    for(i=0;i<n;i++)
    {
        min = 1000;
        for(j=0;j<n;j++)
        {
            if(visited[j] == 0)
            {
                int dist = abs(head - req[j]);
                if(dist < min)
                {
                    min = dist;
                    index = j;
                }
            }
        }
        total += min;
        head = req[index];
        visited[index] = 1;
    }
    printf("Total head movement = %d\n", total);
    return 0;
}
```

```
// ================= EXP 11 =================
// -------- Single Level Directory --------
#include <stdio.h>
#include <string.h>
int main()
{
   char file[20][20], fname[20];
   int i, n = 0, choice, found;
   while(1)
   {
      printf("\n1. Create File");
      printf("\n2. Delete File");
      printf("\n3. Search File");
      printf("\n4. Display Files");
      printf("\n5. Exit");
      printf("\nEnter your choice: ");
      scanf("%d", &choice);
      switch(choice)
      {
         case 1:
            printf("Enter file name: ");
            scanf("%s", file[n]);
            n++;
            printf("File created successfully\n");
            break;
         case 2:
            printf("Enter file name to delete: ");
            scanf("%s", fname);
            found = 0;
            for(i = 0; i < n; i++)
            {
```

```
      if(strcmp(fname, file[i]) == 0)
      {
         for(int j = i; j < n-1; j++)
            strcpy(file[j], file[j+1]);
         n--;
         found = 1;
         printf("File deleted\n");
         break;
      }
   }
   if(found == 0)
      printf("File not found\n");
   break;
case 3:
   printf("Enter file name to search: ");
   scanf("%s", fname);
   found = 0;
   for(i = 0; i < n; i++)
   {
      if(strcmp(fname, file[i]) == 0)
      {
         printf("File found\n");
         found = 1;
         break;
      }
   }
   if(found == 0)
      printf("File not found\n");
   break;
case 4:
   printf("Files in directory:\n");
```

```
            for(i = 0; i < n; i++)
               printf("%s\n", file[i]);
            break;
         case 5:
            return 0;
         default:
            printf("Invalid choice\n");
      }
   }
}
// -------- Multi-Level Directory --------
#include <stdio.h>
#include <string.h>
struct directory
{
   char dname[20];
   char fname[20][20];
   int fcount;
};
int main()
{
   struct directory dir[10];
   int dcount = 0;
   int choice, i, j;
   char dirname[20], filename[20];
   while(1)
   {
      printf("\n1.Create Directory");
      printf("\n2.Create File");
      printf("\n3.Display");
      printf("\n4.Exit");
```

```
printf("\nEnter choice: ");
scanf("%d",&choice);
switch(choice)
{
   case 1:
      printf("Enter directory name: ");
      scanf("%s",dir[dcount].dname);
      dir[dcount].fcount = 0;
      dcount++;
      printf("Directory created\n");
      break;
   case 2:
      printf("Enter directory name: ");
      scanf("%s",dirname);
      for(i=0;i<dcount;i++)
      {
         if(strcmp(dirname,dir[i].dname)==0)
         {
            printf("Enter file name: ");
            scanf("%s",filename);
            strcpy(dir[i].fname[dir[i].fcount],filename);
            dir[i].fcount++;
            printf("File created\n");
            break;
         }
      }
      break;
   case 3:
      printf("\nDirectory Structure:\n");
      for(i=0;i<dcount;i++)
      {
```

```
            printf("\n%s:\n",dir[i].dname);
            for(j=0;j<dir[i].fcount;j++)
            {
               printf(" %s\n",dir[i].fname[j]);
            }
          }
          break;
        case 4:
          return 0;
        default:
          printf("Invalid choice\n");
      }
    }
}
// -------- Sequential Allocation --------
#include <stdio.h>
int main()
{
   int n, i, start, length;
   printf("Enter number of files: ");
   scanf("%d",&n);
   for(i=0;i<n;i++)
   {
      printf("\nEnter starting block and length of file %d: ", i+1);
      scanf("%d %d",&start,&length);
      printf("Allocated blocks: ");
      for(int j=start;j<start+length;j++)
      {
         printf("%d ",j);
      }
   }
```

```
    return 0;
}
// -------- Linked Allocation --------
#include <stdio.h>
int main()
{
    int f[50], i, j, n, start, len, k;
    for(i = 0; i < 50; i++)
        f[i] = 0;
    printf("Enter number of files: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++)
    {
        printf("\nEnter starting block: ");
        scanf("%d", &start);
        if(f[start] == 0)
        {
            printf("Enter number of blocks needed: ");
            scanf("%d", &len);
            printf("Enter the blocks: ");
            for(j = 0; j < len; j++)
            {
                scanf("%d", &k);
                if(f[k] == 0)
                {
                    f[k] = 1;
                    printf("%d -> ", k);
                }
                else
                {
                    printf("Block already allocated\n");
```

```
            break;
          }
        }
        f[start] = 1;
        printf("NULL\n");
      }
      else
      {
        printf("Starting block already allocated\n");
      }
    }
    return 0;
}
// -------- Indexed Allocation --------
#include <stdio.h>
int main()
{
    int f[50], i, j, index, n, k;
    for(i = 0; i < 50; i++)
      f[i] = 0;
    printf("Enter index block: ");
    scanf("%d", &index);
    if(f[index] == 0)
    {
      printf("Enter number of blocks needed: ");
      scanf("%d", &n);
    }
    else
    {
      printf("Index block already allocated\n");
      return 0;
```

```
    }
    printf("Enter the blocks: ");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &k);
        if(f[k] == 0)
        {
            f[k] = 1;
        }
        else
        {
            printf("Block already allocated\n");
            return 0;
        }
    }
    f[index] = 1;
    printf("File allocated successfully\n");
    printf("Index Block %d contains: ", index);
    for(i = 0; i < n; i++)
        printf("%d ", k);
    return 0;
}
```